

JAVASCRIPT FUNCTIONS

Recursive Function va Higher-Order Function

1. Recursive Function

Recursive Function nima?
Recursive Function – bu o'zini o'zi chaqiradigan function.
Ya'ni function ichida yana shu functionning o'zi ishlaydi.
Bu ko'pincha takrorlanadigan masalalarda ishlatiladi.

Sintaksis

```
function name() {  
  name();  
}
```

Muhim qoidasi

Recursive functionda albatta to'xtash sharti bo'lishi kerak.
Bu "Base Case" deyiladi.
Agar base case bo'lmasa function cheksiz ishlaydi va error beradi.

1-misol

```
function count(num) {  
  if (num === 0) {  
    return;  
  }  
  console.log(num);  
  count(num - 1);  
}  
count(5);
```

Tushuntirish

Functionga 5 yuboriladi va u o'zini qayta-qayta chaqiradi:
 $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$.
0 bo'lganda function to'xtaydi.

2-misol — Factorial

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Factorial kodi

```
function factorial(n) {  
  if (n === 1) {  
    return 1;  
  }
```

```
    }  
    return n * factorial(n - 1);  
}  
console.log(factorial(5));
```

Recursive Function qayerda ishlatiladi?

- Factorial
- Fibonacci
- Daraxt strukturalari
- Nested objectlar
- Algoritmalar

Afzalligi

- Kod qisqa bo'ladi
- Murakkab masalalarni osonlashtiradi
- Takroriy ishlarda qulay

Kamchiligi

- Ko'p recursion xotira ishlatadi
- Noto'g'ri yozilsa infinite loop bo'ladi

2. Higher-Order Function

Higher-Order Function – boshqa functionni parameter sifatida oladigan yoki function qaytaradigan function.
Oddiy qilib aytganda:
Function bilan ishlaydigan function.

1-usul — Function parameter sifatida

```
function salom() {  
    console.log("Salom");  
}  
  
function bajar(func) {  
    func();  
}  
  
bajar(salom);
```

Tushuntirish

Bu yerda salom functioni bajar functioniga yuboriladi.
func() ishlaganda aslida salom() ishlaydi.

2-usul — Function qaytarish

```
function creator() {  
    return function() {  
        console.log("Hello");  
    };  
}
```

```
};  
}  
let result = creator();  
result();
```

Mashhur Higher-Order Functionlar

- map()
- filter()
- forEach()
- reduce()

map() misoli

```
let numbers = [1, 2, 3];  
let result = numbers.map(function(num) {  
    return num * 2;  
});  
console.log(result);
```

filter() misoli

```
let numbers = [1, 2, 3, 4, 5];  
let result = numbers.filter(function(num) {  
    return num > 3;  
});  
console.log(result);
```

Higher-Order Function qayerda ishlatiladi?

- Array methodlarda
- React'da
- Node.js'da
- Eventlarda
- API bilan ishlashda

Afzalligi

- Kod toza bo'ladi
- Qayta ishlatish oson
- Katta projectlarda qulay

Kamchiligi

- Boshlanishida tushunish qiyin
- Callback ko'payib ketishi mumkin

Xulosa

Recursive Function

- O'zini o'zi chaqiradi
- Base case kerak
- Takroriy masalalarda ishlatiladi

Higher-Order Function

- Function parameter oladi yoki function qaytaradi
- Callback bilan ishlaydi
- Zamonaviy JavaScriptning muhim qismi